

ShopEasy Reflection Report

Prepared by:
Melisa Özgür
Student ID:
220717716

In this project, each testing technique gave me a different view of the ShopEasy starter code. Specification-based testing was the first technique that clearly revealed real weaknesses. The valid partition and boundary tests for PriceCalculator passed, which showed that the main formula worked correctly for normal inputs. However, when I tested invalid input partitions, I found validation gaps. A negative base price could produce a negative final price, a negative discount acted like a price increase, a discount greater than 100% could make the result negative, and invalid tax rates were accepted. These were not calculation mistakes for valid inputs, but they were important contract problems because the documented valid input domain was not enforced by the implementation.

Structural testing was useful in a different way. The ShoppingCart tests reached 100% branch coverage in JaCoCo, which exceeded the required 80%. This did not reveal a major bug, but it forced me to test paths that are easy to miss, such as adding an already existing product, removing an unknown product, updating a product after skipping a non-matching cart item, and invalid quantity updates. This made me more confident that the cart behavior matched the implementation. At the same time, it also reminded me that high coverage does not automatically prove correctness; it only shows which parts of the code were executed by tests.

Design by Contract directly addressed the validation gaps found earlier. I added Java assert preconditions and postconditions to PriceCalculator and ShoppingCart. After this, invalid inputs were caught as AssertionError when Maven ran tests with assertions enabled using -ea. This was especially useful because it turned assumptions from the Javadocs into executable checks. The main lesson here was that documentation alone is weak unless the program enforces the documented rules.

Property-based testing with jqwik did not reveal new bugs, but it gave stronger confidence than example-based tests alone. The properties checked identity, discount monotonicity, boundedness, cart commutativity, and the fact that applying a discount does not mutate the cart total. Because Task 3 contracts reject invalid inputs, the jqwik generators had to produce only valid values. That was an important design point: property-based testing is powerful, but the generated data must respect the domain; otherwise, tests fail for the wrong reason.

Mocking with Mockito was most useful for testing OrderProcessor. By mocking InventoryService and PaymentGateway, I could isolate the checkout logic without needing real stock or payment

systems. The tests verified the happy path, inventory failure, payment failure, partial quantity unavailable, and especially that `paymentGateway.charge()` is never called when inventory is unavailable. This did not reveal a defect, but it verified an important safety rule: the system should not attempt payment if the order cannot be fulfilled. The limitation is that mocks do not prove that the real inventory or payment integrations work correctly; they only verify how `OrderProcessor` behaves with controlled dependency responses.

The most effective technique per unit of effort was specification-based testing. It was relatively simple to write, but it revealed the most concrete problems in the starter code: invalid inputs were accepted even though the intended input domain excluded them. Structural testing also gave a high return because JaCoCo quickly showed which branches needed more tests. Property-based testing was powerful, but it required more careful generator design. Mocking was valuable because it allowed me to verify interaction behavior, such as ensuring payment is not attempted after inventory failure.

Using the testing pyramid, I would classify this suite mostly as unit-level testing. The specification-based, structural, contract, and property-based tests focus on individual classes such as `PriceCalculator` and `ShoppingCart`. The Mockito tests are also unit tests because the collaborators are replaced with mocks. What is missing is a stronger integration layer: tests where real or fake implementations of inventory and payment work together with `OrderProcessor`. There are also no end-to-end tests for a complete shopping workflow from product creation to checkout using real dependencies.

If I had one more week, I would add integration tests around the checkout flow. I would create simple fake implementations of `InventoryService` and `PaymentGateway` and test realistic scenarios with multiple products, changing stock, successful orders, failed payments, and unavailable items. I would also run PIT mutation testing more seriously to see whether the high JaCoCo coverage actually catches behavioral changes. The final result of `mvn test` was 68 tests run, 0 failures, 0 errors, 0 skipped, and BUILD SUCCESS, so the current suite is stable, but integration and mutation testing would make it stronger.